

AMU | Language, Mind, & Tech  
Programming in Python  
Ali Asgari



Turtle/n



# Programming Languages

To understand why we have so many languages, we must understand that every language is a set of trade-offs. A language that is fast is usually hard to write; a language that is easy to write is often slow. We use these five scales to find the "DNA" of a language:

| Abstraction | Paradigm | Typing | Execution | Domain |
|-------------|----------|--------|-----------|--------|
|-------------|----------|--------|-----------|--------|



**Low-Level:** You are talking to the **hardware**. You must manage memory, electricity, and the CPU yourself. It is extremely fast but takes many lines of code to do simple things.

**High-Level:** You are talking to a **human**. The language uses English-like words and handles the "machine details" for you. It is slower for the computer but much faster for the researcher.

|                    |          |        |           |        |
|--------------------|----------|--------|-----------|--------|
| <b>Abstraction</b> | Paradigm | Typing | Execution | Domain |
|--------------------|----------|--------|-----------|--------|



**Imperative:** You give the computer a **recipe**. You list every single step (*"Open file, find line 1, look for 'X'..."*).

**Declarative:** You describe the **result** you want (*"Give me all words that start with 'X' from this file"*). You don't care how the computer finds it; you just want the answer.

|             |                 |        |           |        |
|-------------|-----------------|--------|-----------|--------|
| Abstraction | <b>Paradigm</b> | Typing | Execution | Domain |
|-------------|-----------------|--------|-----------|--------|



**Static:** You must declare what every piece of data is before you use it (e.g., *"This variable will only ever be a whole number"*). If you try to put a word there, the program won't even start.

**Dynamic:** The language is flexible. It figures out what the data is as it runs. You can change a variable from a "number" to a "word" mid-program. It's faster for prototyping but easier to make mistakes.

|             |          |        |           |        |
|-------------|----------|--------|-----------|--------|
| Abstraction | Paradigm | Typing | Execution | Domain |
|-------------|----------|--------|-----------|--------|



**Compiled:** The entire program is translated into machine code all at once before it runs. This creates a fast, standalone file.

**Interpreted:** The code is read and executed line-by-line by another program. It's like having a live translator at a meeting. It's slower, but you can change the code and see the result instantly.

|             |          |        |                  |        |
|-------------|----------|--------|------------------|--------|
| Abstraction | Paradigm | Typing | <b>Execution</b> | Domain |
|-------------|----------|--------|------------------|--------|



**General Purpose:** You can use it to build anything, a website, a rocket ship, or a linguistic model (e.g., Python, C++).

**Domain Specific (DSL):** These are "Specialists." They are the best in the world at one specific thing but useless for everything else (e.g., SQL for databases, Praat for phonetics).

|             |          |        |           |               |
|-------------|----------|--------|-----------|---------------|
| Abstraction | Paradigm | Typing | Execution | <b>Domain</b> |
|-------------|----------|--------|-----------|---------------|



| Scale              | Spectrum                           | Details / Position                                                                                  |
|--------------------|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Abstraction</b> | <b>Low-Level</b> --- High-Level    | <b>1/10</b> (Deeply tied to physical hardware/CPU architecture)                                     |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Imperative</b> (Direct commands: Move this, Add that)                                            |
| <b>Typing</b>      | -                                  | <b>Untyped</b> (The CPU only sees raw bits/bytes; it doesn't care if they are "words" or "numbers") |
| <b>Execution</b>   | <b>Compiled</b> --- Interpreted    | <b>Assembled</b> (A direct 1:1 translation; technically the fastest "compilation" possible)         |
| <b>Domain</b>      | General --- <b>Domain Specific</b> | <b>System/Hardware</b> (Only used for the "roots" of a computer: Kernels and Drivers)               |





| Scale              | Spectrum                           | Details / Position                                                                  |
|--------------------|------------------------------------|-------------------------------------------------------------------------------------|
| <b>Abstraction</b> | <b>Low-Level</b> --- High-Level    | <b>3/10</b> (The "Portable Assembly"; gives high-level logic with low-level access) |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Imperative</b> (Procedural; focused on functions and step-by-step logic)         |
| <b>Typing</b>      | <b>Static</b> --- Dynamic          | <b>Static</b> (Manual type declarations; checked at compile time)                   |
| <b>Execution</b>   | <b>Compiled</b> --- Interpreted    | <b>Compiled</b> (Turned into a fast, standalone machine-code file)                  |
| <b>Domain</b>      | <b>General</b> --- Domain Specific | <b>General / System</b> (The "Mother" of modern OSs and Python itself)              |



| Scale              | Spectrum                           | Details / Position                                |
|--------------------|------------------------------------|---------------------------------------------------|
| <b>Abstraction</b> | <b>Low-Level</b> --- High-Level    | <b>5/10</b> (Powerful like C, but more organized) |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Multi-Paradigm</b> (Step-by-step + "Objects")  |
| <b>Typing</b>      | <b>Static</b> --- Dynamic          | <b>Static</b> (Very strict rules)                 |
| <b>Execution</b>   | <b>Compiled</b> --- Interpreted    | <b>Compiled</b> (Optimized for maximum speed)     |
| <b>Domain</b>      | <b>General</b> --- Domain Specific | <b>General / Performance</b> (Games and AI)       |



| Scale              | Spectrum                           | Details / Position                                       |
|--------------------|------------------------------------|----------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>8/10</b> (Very friendly to the human writer)          |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Object-Oriented</b> (Strictly organized)              |
| <b>Typing</b>      | <b>Static</b> --- Dynamic          | <b>Static</b> (Safety-first)                             |
| <b>Execution</b>   | <b>Compiled</b> --- Interpreted    | <b>Compiled (JIT)</b> (Safe and portable across Windows) |
| <b>Domain</b>      | <b>General</b> --- Domain Specific | <b>General / Enterprise</b> (Business and VR)            |



| Scale              | Spectrum                           | Details / Position                                                               |
|--------------------|------------------------------------|----------------------------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>8/10</b> (Combines the speed of C with the ease of Python)                    |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Multi-Paradigm</b> (Heavy focus on Protocol-Oriented programming)             |
| <b>Typing</b>      | <b>Static</b> --- Dynamic          | <b>Static &amp; Strong</b> (Known for "Type Inference"—it guesses what you mean) |
| <b>Execution</b>   | <b>Compiled</b> --- Interpreted    | <b>Compiled</b> (Uses the LLVM compiler to be as fast as C++)                    |
| <b>Domain</b>      | General --- <b>Domain Specific</b> | <b>Apple Ecosystem</b> (Used for everything with an "i" in front of it)          |



| Scale              | Spectrum                           | Details / Position                                                          |
|--------------------|------------------------------------|-----------------------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>8/10</b> (Extremely concise; does more with less code)                   |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Object-Oriented + Functional</b><br>(Flexible and modern)                |
| <b>Typing</b>      | <b>Static</b> --- Dynamic          | <b>Static &amp; Strong</b> (Features "Null Safety" by default)              |
| <b>Execution</b>   | <b>Compiled</b> --- Interpreted    | <b>Compiled</b> (Runs on the Java Virtual Machine, making it very portable) |
| <b>Domain</b>      | <b>General</b> --- Domain Specific | <b>Android / Multi-platform</b> (The king of mobile and some backend)       |



| Scale              | Spectrum                           | Details / Position                                              |
|--------------------|------------------------------------|-----------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>7/10</b> (Managed; hides the hardware complexity)            |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Object-Oriented</b> (Strictly organized around "Classes")    |
| <b>Typing</b>      | <b>Static</b> --- Dynamic          | <b>Static &amp; Strong</b> (Very strict; catches errors early)  |
| <b>Execution</b>   | <b>Compiled</b> --- Interpreted    | <b>Compiled (Bytecode)</b> (Runs on the "Java Virtual Machine") |
| <b>Domain</b>      | <b>General</b> --- Domain Specific | <b>General / Enterprise</b> (Server-side and Android)           |



| Scale              | Spectrum                           | Details / Position                                                      |
|--------------------|------------------------------------|-------------------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>9/10</b> (Extremely abstracted from hardware)                        |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Functional + Imperative</b> (Event-driven and flexible)              |
| <b>Typing</b>      | Static --- <b>Dynamic</b>          | <b>Dynamic &amp; Weak</b> (Very loose; "Apple" can become the number 5) |
| <b>Execution</b>   | Compiled --- <b>Interpreted</b>    | <b>Interpreted / JIT</b> (Executed directly by your web browser)        |
| <b>Domain</b>      | <b>General</b> --- Domain Specific | <b>General / Web</b> (The only language browsers understand)            |



| Scale              | Spectrum                           | Details / Position                                                  |
|--------------------|------------------------------------|---------------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>9/10</b> (Far from hardware; purely for file/process management) |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Imperative</b> (Procedural; "Do this, then do that")             |
| <b>Typing</b>      | Static --- <b>Dynamic</b>          | <b>Weakly Typed</b> (Everything is essentially a string of text)    |
| <b>Execution</b>   | Compiled --- <b>Interpreted</b>    | <b>Interpreted</b> (The shell reads and runs it line-by-line)       |
| <b>Domain</b>      | General --- <b>Domain Specific</b> | <b>System Administration / Automation</b>                           |





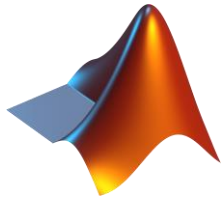
| Scale              | Spectrum                           | Details / Position                                              |
|--------------------|------------------------------------|-----------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>9/10</b> (Built on top of the .NET framework)                |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Object-Oriented</b> (Everything is an object, not just text) |
| <b>Typing</b>      | <b>Static</b> --- Dynamic          | <b>Strongly Typed</b> (Inherits the strictness of C#)           |
| <b>Execution</b>   | Compiled --- <b>Interpreted</b>    | <b>Interpreted / JIT</b> (Runs in the PowerShell environment)   |
| <b>Domain</b>      | General --- <b>Domain Specific</b> | <b>Windows Management / Automation</b>                          |



| Scale              | Spectrum                           | Details / Position                                                                              |
|--------------------|------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>10/10</b> (The highest level; you speak to the data, not the machine)                        |
| <b>Paradigm</b>    | Imperative --- <b>Declarative</b>  | <b>Declarative</b> (You describe the <i>result</i> you want, not the <i>steps</i> to get there) |
| <b>Typing</b>      | <b>Static</b> --- Dynamic          | <b>Static</b> (Database tables have rigid rules for every column)                               |
| <b>Execution</b>   | -                                  | <b>Query-based</b> (The "Database Engine" decides how to execute your request)                  |
| <b>Domain</b>      | General --- <b>Domain Specific</b> | <b>Data Management</b> (The universal language for databases)                                   |



| Scale              | Spectrum                           | Details / Position                                                          |
|--------------------|------------------------------------|-----------------------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>9/10</b> (Designed to feel like mathematical notation)                   |
| <b>Paradigm</b>    | <b>Functional</b> --- Declarative  | <b>Functional</b> (Focuses on data transformations and math functions)      |
| <b>Typing</b>      | Static --- <b>Dynamic</b>          | <b>Dynamic</b> (Flexible; designed for interactive data exploration)        |
| <b>Execution</b>   | Compiled --- <b>Interpreted</b>    | <b>Interpreted</b> (Runs line-by-line; great for "experimenting" with data) |
| <b>Domain</b>      | General --- <b>Domain Specific</b> | <b>Statistics / Data Science</b> (Specifically for math and visualization)  |



| Scale              | Spectrum                           | Details / Position                                                       |
|--------------------|------------------------------------|--------------------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>9/10</b> (Designed to look like Matrix Algebra)                       |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Iterative / Procedural</b> (Built for step-by-step matrix math)       |
| <b>Typing</b>      | Static --- <b>Dynamic</b>          | <b>Dynamic</b> (You don't declare types; everything is a matrix)         |
| <b>Execution</b>   | Compiled --- <b>Interpreted</b>    | <b>Interpreted</b> (Runs in its own dedicated "Desktop" environment)     |
| <b>Domain</b>      | General --- <b>Domain Specific</b> | <b>Engineering / Science</b> (Numerical computing and Signal Processing) |



| Scale              | Spectrum                           | Details / Position                                              |
|--------------------|------------------------------------|-----------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>10/10</b> (Closer to formal logic than machine code)         |
| <b>Paradigm</b>    | Imperative --- <b>Declarative</b>  | <b>Logical</b> (You provide facts/rules; it finds the truth)    |
| <b>Typing</b>      | -                                  | <b>Symbolic</b> (Works with "atoms" and "relations")            |
| <b>Execution</b>   | -                                  | <b>Query-based</b> (Uses an "Inference Engine" to find answers) |
| <b>Domain</b>      | General --- <b>Domain Specific</b> | <b>AI / Linguistics</b> (The king of symbolic AI)               |



| Scale              | Spectrum                           | Details / Position                                            |
|--------------------|------------------------------------|---------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>9/10</b> (Extremely specialized for acoustic data)         |
| <b>Paradigm</b>    | <b>Imperative</b> --- Declarative  | <b>Scripting</b> (Automating the GUI actions of the software) |
| <b>Typing</b>      | Static --- <b>Dynamic</b>          | <b>Weakly Typed</b> (Handles numbers and strings simply)      |
| <b>Execution</b>   | Compiled --- <b>Interpreted</b>    | <b>Interpreted</b> (Runs inside the Praat software)           |
| <b>Domain</b>      | General --- <b>Domain Specific</b> | <b>Phonetics</b> (The global standard for speech analysis)    |



| Scale              | Spectrum                           | Details / Position                                                   |
|--------------------|------------------------------------|----------------------------------------------------------------------|
| <b>Abstraction</b> | Low-Level --- <b>High-Level</b>    | <b>9/10</b> (Reads like English; "Executable Pseudocode")            |
| <b>Paradigm</b>    | <b>Multi-Paradigm</b>              | <b>Multi-Paradigm</b> (Object-Oriented, Imperative, and Functional)  |
| <b>Typing</b>      | Static --- <b>Dynamic</b>          | <b>Dynamic &amp; Strong</b> (Flexible but prevents "illegal" math)   |
| <b>Execution</b>   | Compiled --- <b>Interpreted</b>    | <b>Interpreted</b> (The "Python Interpreter" runs code line-by-line) |
| <b>Domain</b>      | <b>General</b> --- Domain Specific | <b>General / Science / AI</b> (The "Swiss Army Knife" of code)       |



# Python

Coding environment

